

Two-Week Fabric Data Engineering Project

Open data | Medallion architecture | Initial + incremental loading | PySpark | Power BI | Git | Scrum

Project overview

In this four-person team project, you will choose a realistic analytical topic and build a complete Microsoft Fabric data engineering solution. The business topic and detailed technical design are chosen by the team, but the final solution must show a clear path from source data to trusted Gold outputs and Power BI.

The project is deliberately open. A smaller, coherent and repeatable solution is better than a large design that is incomplete by the final presentation.

Core requirement

The solution must support both an initial load and incremental loading. Each run must produce meaningful run-log information, and rerunning the same batch must not create unintended duplicates.

Team setup

- Teams consist of four students.
- Backlog items may be completed by the whole team, individually, or in pairs.
- Every student must make a meaningful technical contribution and be able to explain the overall solution.
- Use Scrum-style planning: Product Backlog, task ownership, status tracking, Definition of Done and regular short stand-ups.
- Use Git throughout the project. Commit code, notebooks, SQL, documentation and architecture material regularly with meaningful messages.

Business scenario and data

- Choose at least two meaningfully related data sources.
- At least one source must be public/open data, an API, a web download or another approved external source.
- The sources must be joinable or enrichable through time, location, category, organization, product, team, country or another meaningful key.
- Write a short problem statement: what questions should the Gold layer and report answer?
- Synthetic or AI-generated data is allowed, but it must be clearly identified and its purpose and limitations documented.

Learning goals

- Select realistic sources and describe their grain, keys, limitations and expected relationship.
- Design a Bronze/Silver/Gold architecture with real quality differences between layers.
- Use PySpark for at least part of ingestion, cleaning, validation, transformation or loading.
- Implement repeatable initial and incremental loading in Fabric.
- Create operational run logging and meaningful validation checks.
- Create a reporting-ready Gold model or mart and a Power BI report that uses Gold outputs.
- Use Git and Scrum practices to manage a small data engineering project.
- Document and present the architecture, implementation, limitations and results.

Accepted Medallion architecture options

Option	Architecture	Notes
1	One schema-enabled Lakehouse	Use bronze, silver and gold schemas.
2	Three Lakehouses	Bronze Lakehouse → Silver Lakehouse → Gold Lakehouse.
3	Lakehouse + Warehouse	Bronze Lakehouse → Silver Lakehouse → Gold Warehouse.

Choose the option that best fits your team and scenario. More Fabric items do not automatically mean a better solution. The team must briefly justify its choice.

Layer expectations

Layer	Expected purpose
Bronze	Raw or near-raw source data. Preserve the original structure where practical and include source metadata such as source/file/API name, batch or run identifier and load timestamp.
Silver	Cleaned, typed, standardized and validated data. Resolve column names, data types, invalid values, duplicates and join keys. Keep rejected/review outputs where useful.
Gold	Reporting-ready fact, dimension or mart tables with a clear grain. Gold must support the selected analytical questions and Power BI report.

Required Fabric work

- Use Fabric notebooks, pipelines, Dataflow Gen2, Lakehouse, Warehouse or other suitable Fabric items as needed.
- Every team member must contribute to PySpark work. SQL, Dataflow Gen2 and other tools may also be used.
- Create a repeatable load process from source to Gold. The exact orchestration design is chosen by the team.
- Power BI must use Gold outputs as its main reporting source, not raw Bronze files or tables.

Initial and incremental loading

The project must demonstrate both an initial load and at least one incremental load. The source does not need to arrive live: a historical public dataset may be divided into an initial batch and one or more later batches, provided the arrival logic is documented.

Requirement	What to demonstrate
Initial load	Load the starting dataset and create the required Bronze, Silver and Gold outputs.
Incremental load	Process only new or changed source data according to a documented strategy.
Rerun behavior	Rerun an already processed batch or load step and show that unintended duplicates are not created.
Control logic	Use a batch ID, file tracking, date range, watermark, source key comparison, MERGE or another justified approach.
Validation	Show row counts, key checks, duplicate checks, date/range checks, unmatched keys or other checks relevant to the scenario.

Run logging

Run logging is mandatory, but teams may design it in a way that fits their solution. A run-log table or equivalent output should make it possible to understand what ran and what happened.

- Run or batch identifier
- Start and finish timestamp
- Status, such as Started, Succeeded or Failed
- Source, entity, file or batch being processed
- Meaningful row counts, such as rows read, inserted, updated or rejected
- Error message or diagnostic information when a run fails

Optional advanced features

- SCD Type 2 using public or synthetic changing master data
- Separate quarantine or rejected-row outputs
- Multiple incremental batches or watermark/control tables
- Dataflow Gen2, Eventstream, Eventhouse, KQL or Real-Time Dashboard
- Additional Gold marts, advanced Power BI measures or automated tests

Optional features should be added only after the mandatory end-to-end solution works.

Git and Fabric collaboration

Because students cannot add other team members to their personal Fabric subscriptions, development may take place in separate Fabric workspaces. The shared Git repository is the main collaboration and version-history mechanism.

- Each student or pair may develop in their own Fabric workspace or subscription.
- Use Fabric Git integration where supported, and keep shared project files in the team repository.
- Use branches, folders, pull requests or another agreed workflow that the team can explain.
- Before the dry run, integrate the required items into one nominated final Fabric workspace.
- The final workspace and repository must represent one coherent, runnable group solution.
- Do not commit passwords, tokens, connection strings with secrets or personal credentials.

Scrum and work distribution

- Create a Product Backlog covering data selection, ingestion, Bronze, Silver, validation, Gold, incremental loading, logging, Power BI, Git, documentation and presentation.
- Give each item an owner or responsible participants and keep its status current.
- Work may be completed together, individually or in pairs. Use the model that best fits the task.
- Agree on a Sprint Goal and Definition of Done.
- Use short daily stand-ups to identify progress, next steps and blockers.
- Each student must be able to show their contribution through backlog items, commits, notebook cells, scripts, pull requests or other evidence.

Suggested project roles or work areas

Possible area	Examples
Sources and Bronze	Source research, API/file ingestion, metadata, batch design
Silver and quality	PySpark cleaning, typing, deduplication, rejected rows, validation
Incremental and operations	Pipeline/orchestration, parameters, control logic, run logging
Gold and reporting	Model grain, dimensions/marts, Power BI, measures and presentation

These are examples only. Teams may divide work differently and may rotate responsibilities.

Minimum deliverables

Deliverable	Expected contents
Source description	Dataset names and links/files, source type, grain, important columns, join idea, limitations and fallback plan.
Product Backlog	PBIs, owners/participants, current status, Sprint Goal and Definition of Done.
Architecture plan	Source → Bronze → Silver → Gold → Power BI, including chosen Fabric items and layer differences.
Fabric implementation	Notebooks, pipelines/Dataflow Gen2 or other Fabric items that implement the solution.
PySpark work	Meaningful PySpark ingestion, cleaning, validation, transformation or loading.
Initial + incremental load	Working initial load, incremental strategy and rerun/duplicate-prevention evidence.
Run logging	Run/batch metadata, status, timestamps and relevant row counts.
Bronze and Silver outputs	Raw/near-raw Bronze and cleaned, typed, validated Silver data.
Gold model or mart	Clear grain, keys and reporting-ready tables.
Power BI report	Useful report pages or views using Gold as the main source.
Git evidence	Repository with code and documentation, meaningful commits and integrated final version.
Runbook and presentation	How to run/refresh the solution and how the team will demonstrate it.

Instructor review checkpoints

Checkpoint	What to show
1. Datasets and problem statement	Selected public, external or generated datasets; source links/files; problem statement; expected join keys and grain; important limitations; and a fallback source if needed.
2. Product Backlog, team split and Git plan	Sprint Goal, Definition of Done, Product Backlog items, owners or responsible participants, planned PySpark contributions, and how work from separate Fabric workspaces will be combined through Git into the final workspace.
3. Medallion, incremental and logging plan	Source → Bronze → Silver → Gold → Power BI plan; chosen Fabric architecture; layer purposes and output names; initial and incremental loading strategy; rerun/duplicate-prevention approach; and the planned run-log fields.
4. Bronze, Silver and quality sanity check	Working source ingestion, Bronze metadata, meaningful PySpark cleaning or transformation, typed and validated Silver outputs, and examples of validation, rejected/review rows or other quality evidence.
5. Gold model and reporting readiness	Gold model or mart, main table grain, keys and relationships, planned Power BI source tables and measures, and validation showing that incremental data reaches the reporting layer correctly.

Wednesday 12.8. dry run / Pitch

The dry run is a practical checkpoint, not the final presentation. The main solution should be substantially complete so that Thursday can be used for corrections and refinement.

- Business problem and selected sources
- Medallion architecture and chosen Fabric items
- Working initial and incremental load
- Run-log records and rerun/duplicate-prevention evidence
- Examples of Bronze, Silver and Gold outputs
- Main Gold grain and key relationships
- Power BI draft
- Git history and Scrum board
- Known limitations and remaining work before Friday

Final presentation

- Briefly explain the scenario, sources, grain and limitations.
- Show the Medallion architecture and what changes between Bronze, Silver and Gold.
- Demonstrate or explain the load process, incremental strategy and run logging.
- Show meaningful PySpark work and at least one validation result.
- Explain the Gold model or mart and show the Power BI report.
- Show Git/Scrum evidence and summarize team contributions.
- Mention limitations and what you would improve with more time.

Project quality checklist

✓	Check
☐	At least two related sources are used; one is public/open/API/download, and grain, keys and limitations are documented.
☐	Bronze, Silver and Gold have clearly different purposes.
☐	PySpark is used in a meaningful project task.
☐	Initial and incremental loading both work.
☐	Rerunning an existing batch does not create unintended duplicates.
☐	Run logging shows status, timestamps and meaningful counts.
☐	Gold has a clear grain and Power BI uses Gold outputs.

☐	Git history, Scrum backlog, final workspace, repository, runbook and presentation form one coherent solution.
--------------------------	---